Internal Website Evaluation Framework – v1 (Complete Documentation)

1. Use Case & Importance

```
Primary Use Case:
Evaluate an internal Website Preview URL generated by the vibe-coding platform and produce:
- Category-wise scores
- Passed / Failed counts
- Critical issue indicators
- Structured findings report
- CI-friendly machine-readable output

Why This Is Important:
1. Deterministic validation of AI-generated code
2. Enables CI/CD quality gates
3. Reduces compliance and security risk
4. Prevents production regressions
5. Provides repeatable and modular evaluation
```

2. Input and Output of the Agent

```
Input:
- Website Preview URL
- Optional evaluation profile (baseline/strict)
- Optional category selection
- CI metadata (commit, branch, PR ID)

API Example:
{
  "url": "https://preview.internal-app/site-xyz",
  "profile": "baseline",
  "categories": ["security", "performance"]
}

Output:
- Category-wise scores
- Passed / Failed counts
- Critical issue indicators
- Structured JSON findings
- CI Markdown summary
- HTML summary report
```

3. Technical Architecture & Tech Stack

Core Stack:
- FastAPI (Evaluation API)
- Python (Orchestrator, Rule Engine, Scoring)
- Redis + RQ (Job Queue)
- Docker (Worker Containerization)
- Playwright (Browser Execution)
- Lighthouse CLI (Performance)
- OWASP ZAP (Security Baseline)
- axe-core (Accessibility)
- PostgreSQL (Metadata + Findings Storage)
- MinIO (Artifact Storage)
- Next.js (Internal Dashboard)

Architecture Flow:
CI/CD Trigger → FastAPI → Orchestrator → Redis Queue → Worker Container
→ Static Analysis Layer → Playwright Execution
→ Dynamic Analysis (ZAP, Lighthouse, axe-core)
→ Rule Engine (YAML deterministic rules)
→ Scoring Engine
→ Evidence Capture (Screenshots, HAR, Reports)
→ PostgreSQL + MinIO
→ Report Generator (JSON + HTML + Markdown)
→ Internal Dashboard

4. Component-Level Breakdown

```
Evaluation API (FastAPI):
- Accepts evaluation request
- Validates schema (Pydantic)
- Generates Run ID
- Enqueues job

Orchestrator:
- Snapshots config
- Tracks job state (PENDING/RUNNING/FAILED/COMPLETED)

Worker (Dockerized):
- Executes static + dynamic scans
- Captures artifacts
- Runs rule evaluation
- Computes scores

Static Analysis Layer:
- Header analysis
- TLS validation
- Dependency scanning
- SEO metadata checks

Dynamic Analysis Layer:
- Playwright automation
- Lighthouse performance audit
- axe-core accessibility scan
- OWASP ZAP baseline scan
- Functional flow validation
- AI/Vibe security pattern checks

Rule Engine:
- YAML-based deterministic rules
- Versioned rule sets
- No probabilistic scoring

Scoring Engine:
- Category weighted scoring
- Critical issue auto-fail
- Pass/Fail counts
- Overall threshold logic

Storage Layer:
- PostgreSQL for metadata & findings
- MinIO for artifacts
- Redis for job state

Reporting Layer:
- Structured JSON output
- Markdown for CI
- HTML for dashboard
```

5. Reliability & Design Principles

- Deterministic rule evaluation
- Modular category-based analyzers
- Containerized reproducible execution
- Timeout and retry boundaries
- Run ID correlation logging
- CI-friendly machine-readable output
- Single-service architecture for v1

6. Phased Roll-out Plan

```
Phase 1 – Foundation & API
- Define Pydantic data models
- Implement FastAPI /evaluate endpoint
- Orchestrator + Redis/RQ
- Worker container scaffold
- Stub YAML rule engine
- Minimal static validation
- Dummy JSON/Markdown output

Phase 2 – Browser & Static Analysis
- Integrate Playwright
- DOM/static analyzers
- Screenshot & DOM evidence capture
- Extend rule engine
- Category scoring aggregation
- Structured JSON + CI Markdown

Phase 3 – Dynamic Analysis Layer
- Lighthouse CLI integration
- axe-core accessibility checks
- OWASP ZAP baseline scan
- Rules consuming tool outputs
- Critical issue detection
- Timeout & retry logic

Phase 4 – Persistence & Artifacts
- PostgreSQL schema design
- SQLAlchemy integration
- MinIO artifact storage
- Persist job lifecycle state
- API endpoints for history

Phase 5 – Reporting & CI Integration
- JSON + Markdown report generator
- CLI helper for pipelines
- Quality gate thresholds
- GitHub Actions / Jenkins examples

Phase 6 – Dashboard & Hardening
- Next.js dashboard
- Drill-down findings view
- OAuth/token authentication
- Worker autoscaling
- Monitoring & metrics
- Unit + integration tests

Decisions:
- YAML-based deterministic rules
- Single-service v1 architecture
- Early containerization for reproducibility
```